

Estrutura de Software — BMS

Battery Management System com TLE9012DQU + TLE9015DQU + STM32G491RET6

Documento Técnico

Rev. 1.3

2026-08-25

Equipe: AMPERA • **Autor:** Guilherme Lettmann

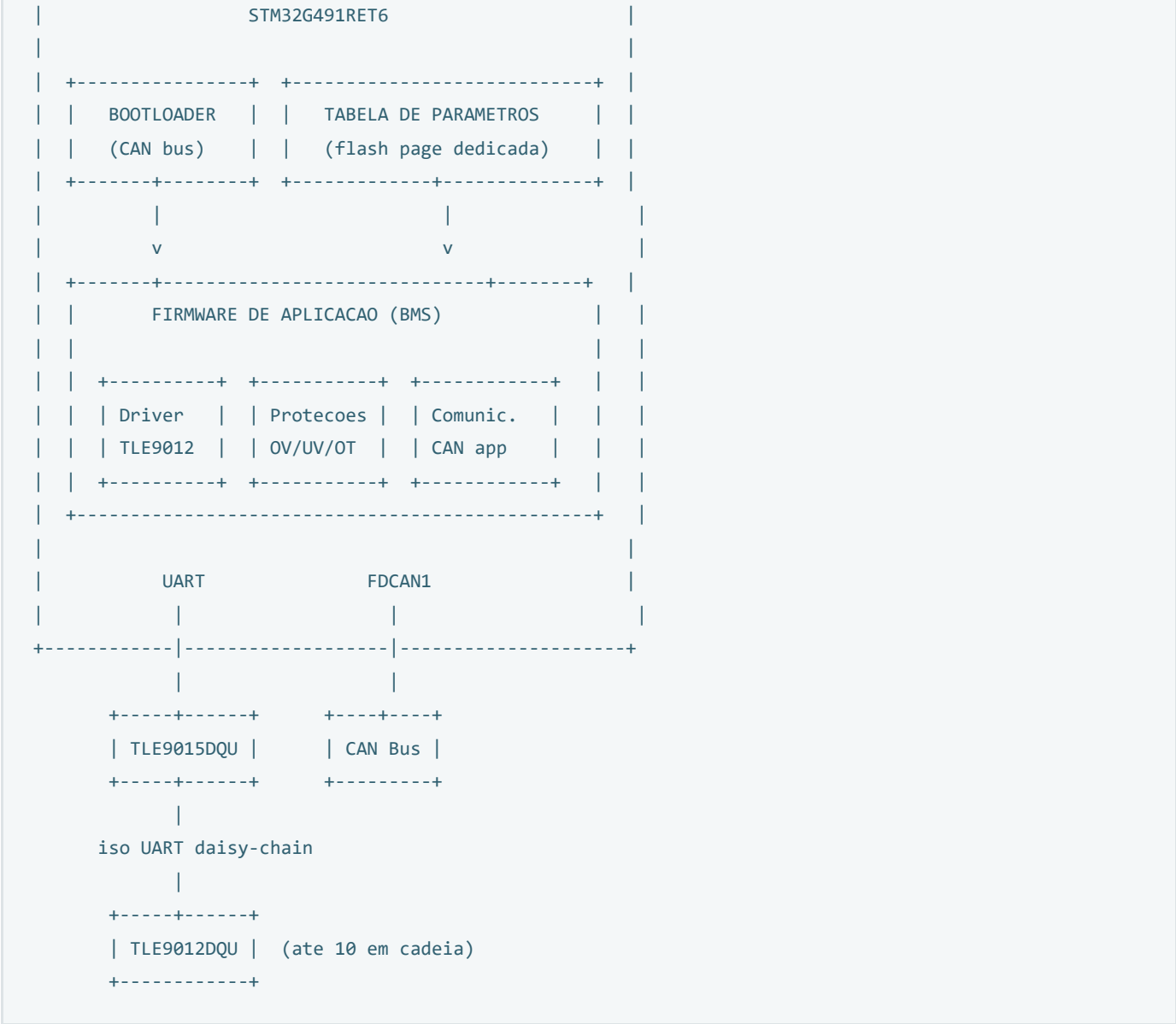
SUMÁRIO

1. Visão Geral da Arquitetura
 1. Hardware de Desenvolvimento
 2. Fases de Implementação
2. Mapa de Memória Flash
3. Bootloader CAN
4. Tabela de Parâmetros
5. Firmware de Aplicação
 1. Módulos
 2. Máquina de Estados
6. Driver TLE9012 / TLE9015
 1. Protocolo iso UART
 2. Estrutura do Frame
 3. CRC SAE-J1850
 4. Sequência de Inicialização
 5. Leitura de Tensões
 6. Leitura de Temperatura
 7. Portabilidade — Isolamento do Transporte
 8. Configuração do Periférico no STM32CubeMX
7. Estrutura de Diretórios do Projeto
8. Fluxo de Atualização em Campo
9. Referências
 1. Uso da Biblioteca de Referência

1. VISÃO GERAL DA ARQUITETURA

O software do BMS é dividido em três camadas independentes, armazenadas em regiões distintas da flash do STM32G491RET6:

+-----+



Objetivo principal: permitir que mudanças de configuração (número de células, thresholds, IDs CAN, etc.) sejam feitas **apenas alterando parâmetros**, sem recompilação. Atualizações de firmware são feitas via **bootloader CAN**, eliminando a necessidade de programador físico.

1.1 Hardware de Desenvolvimento

ITEM	COMPONENTE	OBSERVAÇÃO
MCU	NUCLEO-G491RE	Cortex-M4F @170 MHz, 512 KB flash, 128 KB RAM, 3× FDCAN, ST-LINK integrado
Transceiver	AURIX TLE9015 Adapter Board V2.0	Sinais acessíveis por test points / header COM6 — não exige AURIX
Sensing	BMS Balancing and Sensing Board V6	TLE9012DQU + 12 canais + ladder resistivo on-board
Debug	ST-LINK (on-board)	SWD + VCP para <code>printf</code> pelo mesmo cabo USB

NOTA — ESCOLHA DO MCU

O STM32G491RET6 é equivalente ao STM32G474RET6 para esta aplicação: mesma família STM32G4, mesmo core, mesma flash e mesmo número de FDCAN. A diferença relevante é o HRTIM, presente apenas no G474 e sem uso neste

projeto. Um AURIX TC375 foi considerado (ASIL-D, lockstep, tri-core), mas descartado para a fase de bring-up: a gravação de flash no AURIX é substancialmente mais complexa (bursts de página, rotinas executadas de PSPR, UCBs), o que encarece o bootloader CAN sem benefício funcional para 12 células. A opção permanece aberta — ver seção 6.7.

1.2 Fases de Implementação

O bootloader é a última etapa, não a primeira. Até lá, gravação e depuração são feitas normalmente pelo ST-LINK.

FASE	ESCOPO	ENDEREÇO DO APP	GRAVAÇÃO
1	Driver TLE9012 — ler as 12 tensões via UART	0x0800 0000	ST-LINK (SWD)
2	Proteções, temperatura, balanceamento	0x0800 0000	ST-LINK (SWD)
3	Tabela de parâmetros em flash + CAN da aplicação	0x0800 0000	ST-LINK (SWD)
4	Bootloader CAN	0x0800 4000	ST-LINK ou CAN

NOTA

A migração da fase 3 para a 4 exige apenas duas mudanças no app: `ORIGIN` do linker script para `0x08004000` e reconfiguração do `VTOR`. O ST-LINK continua gravando e depurando o app normalmente mesmo com o bootloader presente na flash — as duas vias de gravação coexistem.

2. MAPA DE MEMÓRIA FLASH

O STM32G491RET6 possui 512 KB de flash. O tamanho de página depende do bit de opção `DBANK` (configuração single-bank ou dual-bank) — consultar RM0440 para o valor efetivo antes de implementar o driver de flash. O layout abaixo usa regiões múltiplas de 4 KB, portanto permanece válido em qualquer uma das duas configurações.

REGIÃO	ENDEREÇO INICIAL	TAMANHO	DESCRIÇÃO
BOOTLOADER	0x0800 0000	16 KB	Bootloader CAN — protegido contra escrita (WRP)
APP	0x0800 4000	480 KB	Firmware de aplicação BMS
PARAMS	0x0807 C000	8 KB	Tabela de parâmetros (página ativa)
PARAMS_BKP	0x0807 E000	8 KB	Tabela de parâmetros (backup)

NOTA

O bootloader deve ser protegido com Write Protection (WRP) via option bytes antes de ir a campo, para evitar sobrescrita acidental. Durante o desenvolvimento (fases 1–3) a WRP fica desabilitada.

ATENÇÃO

Nas fases 1 a 3 o app é compilado no endereço default `0x0800 0000` e ocupa a flash inteira — o mapa acima só passa a valer na fase 4, quando o bootloader é introduzido.

3. BOOTLOADER CAN

O bootloader reside permanentemente na flash e é executado no reset. Ele decide se entra em modo de atualização ou pula para a aplicação.

3.1 Condições de Entrada no Bootloader

- Flag em RAM persistente** — a aplicação seta um valor mágico (ex: `0xDEADBEEF`) num endereço reservado de RAM e executa reset por software. O bootloader lê esse endereço; se encontrar o valor mágico, entra em modo de atualização.
- Aplicação inválida** — o bootloader verifica o CRC32 da região APP armazenado no último word da imagem. Se inválido, permanece no bootloader.
- Comando CAN** — durante uma janela de 200 ms após o reset, o bootloader escuta uma mensagem CAN de solicitação de atualização.

3.2 Protocolo de Atualização

PASSO	DIREÇÃO	CAN ID	CONTEÚDO
1	PC → MCU	0x600	Comando ENTER_BOOTLOADER
2	MCU → PC	0x601	ACK + versão do bootloader
3	PC → MCU	0x600	ERASE_APP (apaga região APP)
4	MCU → PC	0x601	ACK após erase completo
5	PC → MCU	0x602	DATA — pacotes de 8 bytes sequenciais
6	MCU → PC	0x601	ACK a cada página (4 KB) gravada
7	PC → MCU	0x600	VERIFY — CRC32 da imagem completa
8	MCU → PC	0x601	ACK/NACK — resultado da verificação
9	PC → MCU	0x600	JUMP_APP — reset para a aplicação

ATENÇÃO

O FDCAN1 do STM32G491 é usado tanto pelo bootloader quanto pela aplicação. O bootloader configura o CAN com bitrate fixo de 500 kbit/s. A aplicação pode reconfigurar conforme a tabela de parâmetros.

3.3 Salto para a Aplicação

Após validar a imagem, o bootloader:

- Desabilita todas as interrupções
- Desinicializa periféricos usados (FDCAN, SysTick)
- Configura o MSP (Main Stack Pointer) com o valor do primeiro word em `0x0800 4000`
- Salta para o endereço do Reset_Handler em `0x0800 4004`

REQUISITO

O linker script da aplicação deve ter `FLASH (rx) : ORIGIN = 0x08004000` e o VTOR deve ser reconfigurado no `SystemInit` para `0x08004000`.

4. TABELA DE PARÂMETROS

Todos os valores configuráveis do BMS ficam numa struct armazenada nas últimas páginas da flash. A aplicação lê essa struct no boot e nunca usa constantes hard-coded.

PARÂMETRO	TIPO	DEFAULT	UNIDADE	DESCRIÇÃO
num_cells	<code>uint8</code>	12	—	Número de células por TLE9012
num_slaves	<code>uint8</code>	1	—	Número de TLE9012 na cadeia
ov_threshold_mV	<code>uint16</code>	4200	mV	Threshold de sobretensão
uv_threshold_mV	<code>uint16</code>	2800	mV	Threshold de subtensão
ot_threshold_C	<code>int16</code>	60	°C	Threshold de sobretemperatura
ut_threshold_C	<code>int16</code>	-20	°C	Threshold de subtemperatura
bal_enable	<code>uint8</code>	1	bool	Habilita balanceamento passivo
bal_delta_mV	<code>uint16</code>	10	mV	Delta para iniciar balanceamento
can_bitrate	<code>uint32</code>	500000	bit/s	Bitrate do CAN da aplicação
can_base_id	<code>uint16</code>	0x100	—	ID base das mensagens CAN TX
meas_period_ms	<code>uint16</code>	100	ms	Período de medição de tensão
temp_period_ms	<code>uint16</code>	1000	ms	Período de medição de temperatura
num_ntc	<code>uint8</code>	5	—	Número de sensores NTC por TLE9012
uart_baudrate	<code>uint32</code>	2000000	baud	Baudrate da iso UART — mínimo 1000000
crc32	<code>uint32</code>	—	—	CRC32 da struct (validação)

NOTA

Os parâmetros podem ser atualizados via mensagem CAN sem necessidade de bootloader — a aplicação recebe o comando, apaga a página de parâmetros, grava a nova struct e reinicia. O campo `crc32` protege contra corrupção; se inválido, a aplicação carrega os valores default.

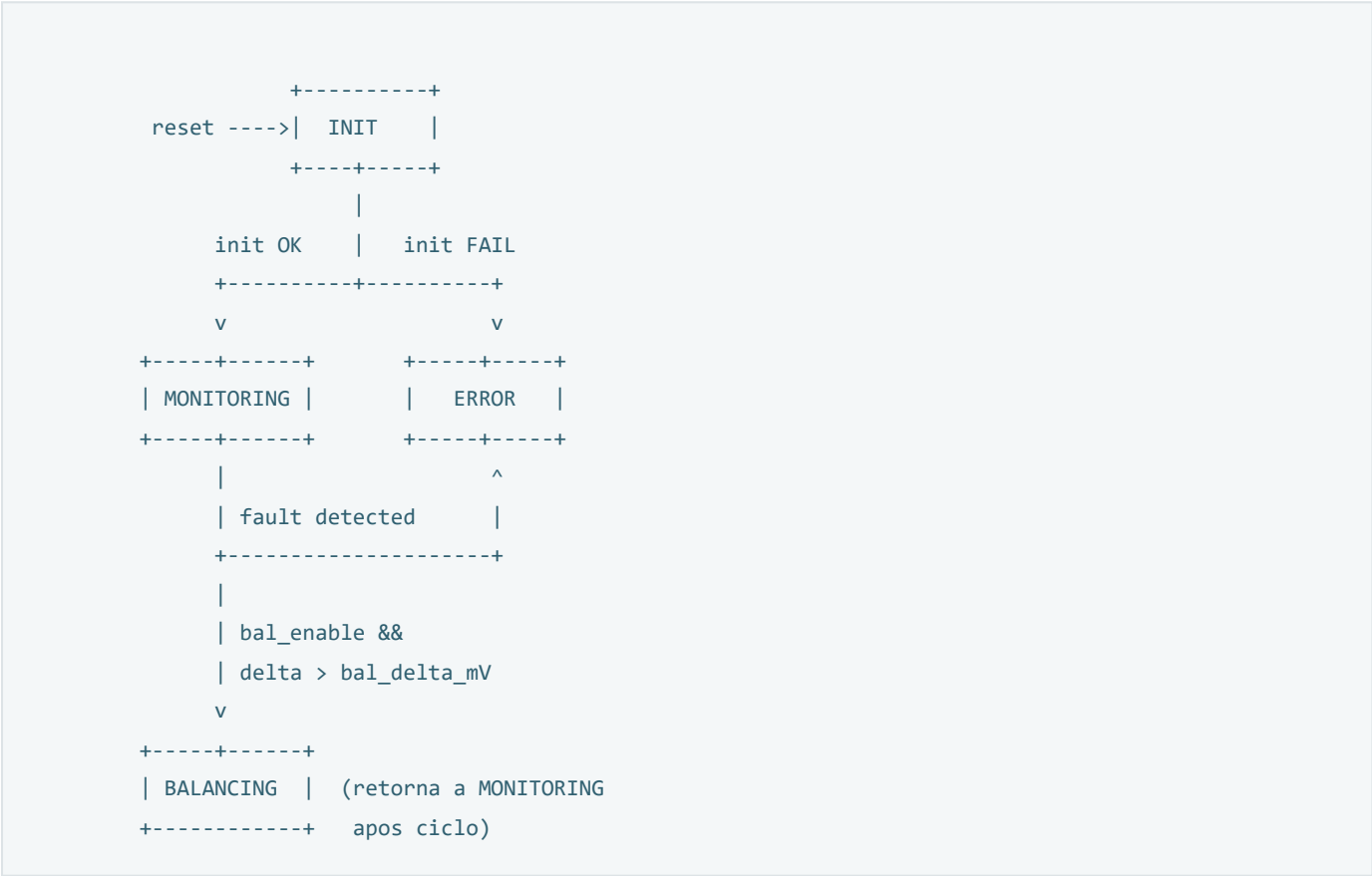
5. FIRMWARE DE APLICAÇÃO

5.1 Módulos

MÓDULO	ARQUIVO	RESPONSABILIDADE
Main	<code>main.c</code>	Init de periféricos, loop principal, scheduler

MÓDULO	ARQUIVO	RESPONSABILIDADE
TLE9012 Driver	t1e9012.c/h	Comunicação iso UART, leitura de células e temperaturas, balanceamento
TLE9015 Driver	t1e9015.c/h	Init do transceiver, wake-up, sleep
CRC	crc_j1850.c/h	CRC SAE-J1850 (lookup table) para frames iso UART
Parâmetros	params.c/h	Leitura/escrita da tabela de parâmetros na flash
Proteções	protection.c/h	Lógica OV, UV, OT, UT, open-wire — flags e ações
Balanceamento	balancing.c/h	Algoritmo de balanceamento passivo baseado em delta de tensão
CAN App	can_app.c/h	Transmissão de dados, recepção de comandos, atualização de parâmetros
BSP	bsp.c/h	Abstração de GPIOs, LEDs, ERR pin

5.2 Máquina de Estados



- INIT:** configura periféricos, carrega parâmetros, inicializa cadeia TLE9012 (atribui NODE_IDs), configura registradores (PART_CONFIG, OV/UV thresholds, TEMP_CONF).
- MONITORING:** loop periódico — dispara medição (PCVM_START), lê 12 tensões e temperaturas, verifica proteções, transmite via CAN.
- BALANCING:** ativa balanceamento passivo nas células acima da tensão mínima + delta. Desativa durante medição (PBOFF).
- ERROR:** desativa balanceamento, sinaliza via CAN e ERR pin. Permite retry após comando CAN.

6. DRIVER TLE9012 / TLE9015

6.1 Protocolo iso UART

A comunicação entre o MCU e o TLE9015 é feita por UART padrão. O TLE9015 converte para iso UART (diferencial, half-duplex, 2 fios com acoplamento capacitivo ou por transformador) e encaminha para a cadeia de TLE9012.

PARÂMETRO	VALOR
Baudrate	2 Mbit/s (oficial) — mínimo 1 Mbit/s
Data bits	8
Paridade	Nenhuma
Stop bits	1
Direção	Single-wire half-duplex (TX e RX no mesmo pino)

REQUISITO CRÍTICO — BAUDRATE

A iso UART **não funciona abaixo de 1 Mbit/s**. O baudrate oficialmente suportado é 2 Mbit/s. Os 115200 baud que aparecem na Figura 14 do manual da Infineon referem-se ao **terminal de debug PC↔AURIX** (TeraTerm), *não* ao link iso UART — confundir os dois leva a um link que jamais estabelece comunicação. O próprio menu do firmware de demo confirma (`CS 1000000 → UART baudrate change to 1Mb`).

REQUISITO CRÍTICO — TIMING ENTRE BYTES

Os bytes de um frame devem ser transmitidos **sem atraso significativo entre si**, sob pena de disparar timeout no transceiver iso UART e corromper a mensagem. A 2 Mbit/s cada byte dura 5 μ s (10 bits), o que inviabiliza envio byte-a-byte por polling. **Consequência de projeto: o TX deve usar DMA**, entregando os 6 bytes do frame como um bloco contíguo ao USART. O RX também deve usar DMA (circular) para não perder bytes.

NOTA — VIABILIDADE NO G491

A 170 MHz, 2 Mbit/s exige divisor `BRR = 85` no USART1 — valor inteiro exato, sem erro de baudrate. É justamente essa exigência de timing que inviabiliza Arduinos baseados em ATmega328 nesta aplicação, e que o Cortex-M4 com DMA atende sem dificuldade.

Ligação física MCU → TLE9015

Pelo esquemático do transceiver (Figura 24 do manual), `UART_LS` é um **pino único**: o host conecta TX e RX a ele através da rede `R_UART` (1,5 k Ω típico, $\tau \leq 50$ ns — Tabela 5). Trata-se portanto de **single-wire half-duplex**, e não de um par TX/RX independente.

A biblioteca de referência `TLE9012_Arduino_Lib` (ver seção 9) confirma independentemente a mesma topologia: “cada pino funciona como RX e TX”, sendo necessário “conectar os pinos rx e tx através de um resistor série”, com valor típico de 1 k Ω a 1,5 k Ω — coerente com o `R_UART` da Tabela 5.

ATENÇÃO

Duas opções de implementação no STM32: (a) modo single-wire nativo do USART pelo bit `HDSEL` (`HAL_HalfDuplex_Init`), em que TX e RX compartilham o pino `TX` em open-drain; ou (b) manter TX e RX

separados no MCU e uni-os externamente pelo resistor série, como fazem as bibliotecas Arduino. A opção (a) dispensa o resistor no lado do MCU, mas exige descartar o eco local de cada byte transmitido. Confirmar a topologia da placa antes de fiar.

SINAL (ADAPTER BOARD)	NUCLEO-G491RE	FUNÇÃO
UART_LS	PA9 — USART1_TX	iso UART low-side (single-wire, via R_UART)
nSLEEP	GPIO livre	Wake-up do transceiver
ERRQ	GPIO (EXTI)	Sinalização de erro do transceiver
3.3V / GND	3V3 / GND	Referência comum — lado de baixa tensão

NOTA

O USART2 (PA2 / PA3) do NUCLEO-64 está ligado ao VCP do ST-LINK por solder bridges; reservá-lo para `printf` de debug permite acompanhar as tensões pelo mesmo cabo USB da gravação. Por isso o TLE9015 usa o USART1. A tabela acima é uma proposta de pinagem, não uma restrição do hardware.

REQUISITO

A NUCLEO-G491RE **não possui transceiver CAN on-board**. Para as fases 3 e 4 é necessário um transceiver externo (ex.: TJA1051, SN65HVD230) ligado aos pinos do FDCAN1, com terminação de 120 Ω.

6.2 Estrutura do Frame

O frame de **escrita** tem 6 bytes; o de **leitura** tem 4 (não possui campo DATA):

BYTE	CAMPO	WRITE	READ	DESCRIÇÃO
0	SYNC	✓	✓	Sempre 0x1E — sincronização
1	ID	✓	✓	bit[7]: R/W (1=write, 0=read); bits[5:0]: NODE_ID (0x00=não endereçado, 0x3F=broadcast)
2	ADDR	✓	✓	Endereço do registrador alvo
3–4	DATA	✓	—	Dado, big-endian. Ausente no frame de leitura
5	CRC8	✓	✓	CRC SAE-J1850 sobre os bytes anteriores

CORREÇÃO EM RELAÇÃO ÀS REVISÕES 1.0–1.2

As revisões anteriores descreviam o frame de leitura como tendo 6 bytes, com o host enviando 0x0000 no campo DATA. Está **incorreto**: o frame de leitura tem 4 bytes. Confere com a Tabela 11 do manual (que lista apenas SYNC / ID / ADDR / CRC) e com a contagem de bytes da biblioteca de referência.

Ordem de bits — MSB first

Os bytes trafegam com o **bit mais significativo primeiro**, ao contrário do UART convencional. A biblioteca Arduino contorna isso invertendo os bits por software (`msb_first_converter`) porque o core do Arduino só transmite LSB-first.

CR2.MSBFIRST

O USART do STM32 implementa MSB-first em **hardware**, tanto no TX quanto no RX. No CubeMX: USART1 → Parameter Settings → Advanced Parameters → *MSB First* = *Enable*. Dispensa completamente a inversão de bits por software.

Eco e contagem de bytes

Como a linha é compartilhada, o host **recebe de volta tudo o que transmite**. O driver deve descartar o eco antes de interpretar a resposta:

OPERAÇÃO	HOST ENVIA	TOTAL RECEBIDO	COMPOSIÇÃO
Read	4 bytes	9 bytes	4 de eco + 5 de resposta
Write	6 bytes	7 bytes	6 de eco + 1 byte de reply frame

6.3 CRC — dois polinômios distintos

O protocolo usa **dois** CRCs diferentes, um para frames de dados e outro para o reply frame:

CRC	APLICAÇÃO	POLINÔMIO	INIT	XOR FINAL	VALIDAÇÃO
CRC8	Frames de dados (read e write)	0x1D	0xFF	0xFF	Comparar com o byte recebido
CRC3	Reply frame do write (1 byte)	0xB0	o próprio byte	—	Válido se o resultado for zero

O CRC8 é o SAE-J1850: $G(z) = z^8 + z^4 + z^3 + z^2 + 1$. Implementar por lookup table de 256 bytes — a 2 Mbit/s o orçamento de tempo por frame é apertado.

O CRC3 opera sobre o único byte de reply, deslocando 5 vezes com realimentação condicional do polinômio **0xB0**; o frame é considerado válido quando o resultado é zero.

NOTA

As revisões 1.0–1.2 documentavam apenas o CRC8. O CRC3 do reply frame foi identificado na implementação de referência (seção 9.1) e é obrigatório para validar confirmações de escrita.

6.4 Sequência de Inicialização

A inicialização atribui NODE_IDs sequencialmente, começando pelo IC mais próximo ao transceiver:

PASSO	AÇÃO	FRAME (HEX)
1	Atribuir NODE_ID = 0x01 ao 1º IC	1E 80 36 0001 ED
2	Atribuir NODE_ID = 0x02 ao 2º IC	1E 80 36 0002 CA
3	...	...
N	Atribuir NODE_ID = N ao último IC (FN=1)	1E 80 36 08xx CRC

NOTA

O último IC da cadeia deve ter o bit FN (Final Node) setado no campo DATA (bit 11 = 1, ex: `0x0804` para `NODE_ID=4`). Sem isso, o broadcast gera timeout na iso UART. O watchdog do TLE9012 deve ser alimentado durante a inicialização, caso contrário o IC volta para sleep e perde o `NODE_ID`.

6.5 Leitura de Tensões

1. Configurar `PART_CONFIG` (0x01) com as células ativas: `0x0FFF` para 12 células
2. Escrever `MEAS_CTRL` (0x18) com `PCVM_START=1`, `CVM_MODE=16-bit`: `0xE021`
3. Aguardar tempo de conversão (~2 ms para 16-bit)
4. Ler registradores `PCVM_0` (0x19) a `PCVM_11` (0x24)
5. Converter: $V_{cell} [V] = (5.0 / 65536) \times RESULT$

FÓRMULA DE CONVERSÃO

$$V = (FSR / 2^{16}) \times RAW$$

Onde `FSR` = 5,0 V (Full Scale Range do ADC do TLE9012). Para `PCVM_0 = 0xABCD`: $V = (5,0 / 65536) \times 0xABCD = 3,355 \text{ V}$.

6.6 Leitura de Temperatura

O TLE9012 suporta até 5 sensores NTC externos, configurados via `TEMP_CONF` (0x04). Os resultados ficam em `TEMP_RESULT_0` (0x29) a `TEMP_RESULT_4` (0x2D). A conversão depende do NTC utilizado (tabela de lookup específica do componente).

6.7 Portabilidade — Isolamento do Transporte

O módulo `tle9012.c` não deve conter **nenhuma chamada de HAL**. Toda a dependência de hardware é injetada por uma interface de transporte, o que mantém o driver — montagem de frame, CRC, mapa de registradores, conversões — independente do MCU:

```
typedef struct {
    bool (*send)(const uint8_t *buf, uint16_t len);
    bool (*recv)(uint8_t *buf, uint16_t len, uint32_t timeout_ms);
    void (*delay_us)(uint32_t us);
} tle9012_transport_t;
```

Consequência prática: migrar para um AURIX TC375 no futuro exige reimplementar apenas essas três funções sobre ASCLIN, sem tocar na lógica do driver. O mesmo vale para testes unitários em PC, onde o transporte pode ser substituído por um *mock* que reproduz respostas conhecidas do TLE9012.

6.8 Configuração do Periférico no STM32CubeMX

USART1 — Parameter Settings

CAMPO	VALOR	JUSTIFICATIVA
Mode	Asynchronous	TX PA9 , RX PA10 — casa com os pads LS_TX/LS_RX da adapter board
Baud Rate	2000000	Mínimo funcional é 1 Mbit/s; 2 Mbit/s é o oficial
Word Length	8 Bits	Sem paridade
Parity / Stop Bits	None / 1	—
Data Direction	Receive and Transmit	—
Over Sampling	16	A 170 MHz resulta em BRR = 85 exato
Advanced → MSB First	Enable	Obrigatório — protocolo é MSB-first (seção 6.2)
Advanced → Auto Baudrate	Disable	—
Advanced → TX/RX Swap	Disable	—

Alternativa: modo *Single Wire (Half-Duplex)*, que usa apenas **PA9** em open-drain bidirecional. Nesse caso o resistor série fica por conta do projetista. Ambos os modos produzem eco — a diferença é apenas o número de fios.

USART1 — DMA Settings

REQUEST	DIRECTION	MODE	PRIORITY	DATA WIDTH
USART1_TX	Memory → Peripheral	Normal	High	Byte / Byte
USART1_RX	Peripheral → Memory	Circular	Very High	Byte / Byte

Increment Address: apenas **Memory** marcado (default do CubeMX, correto).

NOTA — POR QUE CIRCULAR NO RX

O protocolo é request/response e o host recebe o próprio eco. Em modo *Normal*, o DMA precisaria ser re-armado entre frames, e a 2 Mbit/s a resposta do escravo pode chegar exatamente dentro dessa janela. Em modo *Circular* a recepção nunca para, eliminando a janela de perda. O DMAMUX do STM32G4 roteia os requests automaticamente — nada a configurar manualmente.

NVIC Settings

- **USART1 global interrupt: Enable** — necessário para detecção de linha ociosa (`HAL_UARTEx_ReceiveToIdle_DMA`), que delimita o fim do frame
- **DMA1 channelX global interrupt** — habilitados automaticamente ao adicionar os requests; apenas conferir

Demais periféricos

ITEM	CONFIGURAÇÃO	FASE
Clock	HSI16 → PLL (M=4, N=85, R=2) = 170 MHz ; Flash 4WS + PWR boost mode	1

ITEM	CONFIGURAÇÃO	FASE
USART2	PA2 / PA3, 115200, assíncrono, sem DMA — VCP do ST-LINK para <code>printf</code>	1
Watchdog do TLE9012	Sem periférico dedicado — SysTick + timer por software	1
FDCAN1	Adiar; exige transceiver externo	3

ATENÇÃO

O watchdog interno do TLE9012 **reseta o NODE_ID** se não for alimentado dentro do intervalo configurado (WDOG_CNT.WD_CNT), inclusive durante a própria inicialização da cadeia. O tick de *watchdog kick* precisa existir desde a fase 1, antes mesmo de qualquer leitura de tensão.

NOTA — CAMINHO PARA ASIL

Caso o BMS evolua para o AMS definitivo do veículo e seja exigido nível de integridade de segurança (lockstep, ASIL-D), o AURIX TC375 volta a ser candidato. A decisão pode ser adiada sem custo desde que o isolamento de transporte descrito acima seja respeitado desde a fase 1.

7. ESTRUTURA DE DIRETÓRIOS DO PROJETO

```
bms_tle9012/
+-- docs/
|   +-- relatorio_estrutura_software_bms.html
|   +-- relatorio_estrutura_software_bms.pdf
|   +-- infineon-...-usermanual-en.pdf
|
+-- bootloader/                                (projeto STM32CubeIDE separado)
|   +-- Core/Src/
|       |   +-- main.c                        bootloader main
|       |   +-- can_boot.c/h                  protocolo CAN de atualizacao
|       |   +-- flash_if.c/h                  interface flash (erase/write/verify)
|       +-- STM32G491RETX_FLASH.ld            linker: ORIGIN = 0x08000000, LENGTH = 16K
|
+-- app/                                        (projeto STM32CubeIDE principal)
|   +-- Core/Src/
|       |   +-- main.c
|       |   +-- tle9012.c/h
|       |   +-- tle9015.c/h
|       |   +-- crc_j1850.c/h
|       |   +-- params.c/h
|       |   +-- protection.c/h
|       |   +-- balancing.c/h
|       |   +-- can_app.c/h
|       |   +-- bsp.c/h
|       +-- STM32G491RETX_FLASH.ld            linker: fases 1-3 -> ORIGIN = 0x08000000
|                                               fase 4 -> ORIGIN = 0x08004000, LENGTH = 480K
|
+-- tools/
```

<code>+++ flash_tool.py</code>	script Python para envio de firmware via CAN
<code>+++ param_tool.py</code>	script Python para configurar parametros via CAN

8. FLUXO DE ATUALIZAÇÃO EM CAMPO

8.1 Atualização de Firmware (via bootloader CAN)

1. Conectar adaptador CAN-USB ao barramento (ex: PCAN-USB, CANable)
2. Executar `flash_tool.py --port CAN0 --file app.bin`
3. O script envia comando ENTER_BOOTLOADER → a aplicação seta flag em RAM e reseta
4. Bootloader apaga, grava e verifica a nova imagem
5. MCU reinicia na nova aplicação

8.2 Atualização de Parâmetros (sem bootloader)

1. Executar `param_tool.py --port CAN0 --config params.json`
2. O script serializa a struct e envia via mensagens CAN segmentadas
3. A aplicação grava na flash e reinicia com os novos valores

VANTAGEM

Zero recompilação

Para mudar de 12 células para 8, ou de OV 4,2 V para 4,15 V, basta editar o `params.json` e rodar o script. Não é necessário IDE, compilador ou programador.

9. REFERÊNCIAS

1. Infineon, *TLE9012DQU, TLE9015DQU — User Manual*, Rev. 1.0, Z8F80064984, 2022-01-24.
2. STMicroelectronics, *STM32G4 Series — Reference Manual*, RM0440 (cobre a linha G491).
3. STMicroelectronics, *STM32G491xC/xE — Datasheet*.
4. STMicroelectronics, *NUCLEO-G491RE — User Manual* (pinagem, solder bridges do VCP).
5. SAE J1850, *Class B Data Communications Network Interface* — CRC polynomial.
6. STMicroelectronics, *AN2606 — STM32 system memory boot mode*.
7. MaxMax-embedded, *TLE9012_Arduino_Lib*, licença MIT.

https://github.com/MaxMax-embedded/TLE9012_Arduino_Lib

9.1 Uso da Biblioteca de Referência

A biblioteca *TLE9012_Arduino_Lib* é uma implementação aberta e funcional do driver, validada em ESP32 e Arduino Uno R4. Ela não é portátil diretamente (depende de `HardwareSerial`), mas serve como referência de alto valor em três pontos:

1. **Mapa de registradores** — `TLE9012_Makros.h` já contém endereços e máscaras de bitfield transcritos do datasheet, eliminando a etapa mais tediosa e mais sujeita a erro do desenvolvimento.

2. **Modelo de API** — a separação entre acesso de baixo nível (`readRegisterSingle` , `writeRegisterBroadcast`) e funções de alto nível (`readCellVoltages` , `startBalancing`) é um bom ponto de partida para a interface do nosso `t1e9012.c` .
3. **Tratamento de erros por callback** — `attachErrorHandler()` com ponteiros de função por tipo de falha, aderente à abordagem de transporte injetável da seção 6.7.

NOTA — LICENÇA

A licença MIT permite reutilização e modificação, inclusive em projeto fechado, desde que o aviso de copyright e o texto da licença sejam preservados. Se trechos forem portados, manter a atribuição no cabeçalho do arquivo correspondente.

LIMITAÇÕES CONHECIDAS DA REFERÊNCIA

A própria biblioteca documenta que o comando *multiread* não está implementado nas versões atuais. Como o multiread é o mecanismo mais eficiente para ler as 12 tensões num único ciclo, essa é uma funcionalidade que precisará ser escrita do zero — ver registradores MULTI_READ (0x24) e MULTI_READ_CFG (0x25) no manual.